NEW FEATURE / QUICK START

LaTeX pseudocode in LyX

Write actual LaTeX commands. LyX shows their compiled result when you leave the source block, and exports the same code to PDF. Line numbers, keywords, indentation and block guides are supplied by `algpseudocodex`.

Try it in the editor

1. Open `demo/pseudocode-demo.lyx` in the embedded LyX editor.
2. Put the cursor in a text paragraph, or inside an Algorithm float after its caption.
3. Open the environment dropdown (env), type `LaTeX Pseudocode`, and press Enter. An editable LaTeX block appears.
4. Replace the example commands with your own. Keep one matching pair of `algorithmic` begin/end commands and the enclosing `minipage` wrapper.
5. Click in a paragraph outside the block, or choose `Render LaTeX Pseudocode` from env. The preview appears after compilation.
6. Click the preview to return to the source, then click inside its red LaTeX box to edit. Use ordinary LyX Save and Undo.

A first working example

The following is the algorithmic part of a block. The env command supplies its wrapper automatically.

```
\begin{algorithmic}[1]
\State $k\gets 0$
\While{$k<3$}
  \State $k\gets k+1$
\EndWhile
\end{algorithmic}
```

Rendered from exactly that source

1: $k \leftarrow 0$
2: **while** $k < 3$ **do**
3: $\quad k \leftarrow k + 1$
4: **end while**

What changes: a block is real LaTeX inside a native LyX Preview inset. Source indentation is optional; the paired commands define the displayed nesting. Existing Enumerate algorithms are not converted automatically.

EXAMPLE 1 / SOURCE AND OUTPUT

Nested While and For loops

Choose `LaTeX While` or `LaTeX For` from env to insert a complete, paired block. Inside an existing source block, put the caret before the enclosing end command before adding the inner loop.

Copy these lines inside the algorithmic environment

```
\Require $x_0\in\mathbb{R}^n$, $K>0$, $\alpha>0$, $\varepsilon>0$
\State $x\gets x_0$, $k\gets 0$
\While{$\|g(x)\|>\varepsilon$ and $k<K$}
  \For{$i=1,\ldots,n$}
    \State $x_i\gets x_i-\alpha g_i(x)$
  \EndFor
  \State $k\gets k+1$
\EndWhile
\State \Return $x$
```

Rendered result

Require: $x_0 \in \mathbb{R}^n$, $K > 0$, $\alpha > 0$, $\varepsilon > 0$
1: $x \leftarrow x_0$, $k \leftarrow 0$
2: **while** $\|g(x)\| > \varepsilon$ and $k < K$ **do**
3: **for** $i = 1, \ldots, n$ **do**
4: $x_i \leftarrow x_i - \alpha g_i(x)$
5: **end for**
6: $k \leftarrow k + 1$
7: **end while**
8: **return** x

Nesting rule: close inner blocks first. Each `\State` creates a numbered statement. `\Require` and `\Ensure` create input/output labels without line numbers.

EXAMPLE 2 / SOURCE AND OUTPUT

If, ElseIf, Else and Return

Use `LaTeX If / Else` and `LaTeX Function` as starting points. Add `\ElsIf` in the source for additional branches. The exact spelling is `ElsIf`.

Copy these lines inside the algorithmic environment

```
\Function{Clip}{$x,\ell,u$}
  \If{$x<\ell$}
    \State \Return $\ell$ \Comment{Lower bound}
  \ElsIf{$x>u$}
    \State \Return $u$ \Comment{Upper bound}
  \Else
    \State \Return $x$
  \EndIf
\EndFunction
\State $z\gets\Call{Clip}{y,0,1}$
```

Rendered result

1: **function** CLIP(x, ℓ, u)
2: ▏ **if** $x < \ell$ **then**
3: ▏ ▏ **return** ℓ ▷ *Lower bound*
4: ▏ **else if** $x > u$ **then**
5: ▏ ▏ **return** u ▷ *Upper bound*
6: ▏ **else**
7: ▏ ▏ **return** x
8: ▏ **end if**
9: **end function**
10: $z \leftarrow$ CLIP($y, 0, 1$)

Return rule: write `\State \Return ...` for a numbered return line. `\Comment{...}` attaches an explanation to the current statement.

EXAMPLE 3 / SOURCE AND OUTPUT

Repeat, Procedure and ForAll

Choose `LaTeX Repeat / Until`, `LaTeX Procedure`, or `LaTeX For All`.

Repeat closes with `\Until{condition}`. ForAll closes with `\EndFor`.

Copy these lines inside the algorithmic environment

```
\Procedure{Refine}{$x_0,\varepsilon$}
  \State $x\gets x_0$
  \Repeat
    \State $x_{\mathrm{old}}\gets x$
    \State $x\gets T(x)$ \Comment{One iteration}
  \Until{$\|x-x_{\mathrm{old}}\|\leq\varepsilon$}
  \State \Return $x$
\EndProcedure
\ForAll{$x_0\in S$}
  \State \Call{Refine}{$x_0,\varepsilon$}
\EndFor
```

Rendered result

procedure REFINE(x_0, ε)
    $x \leftarrow x_0$
    repeat
        $x_{\text{old}} \leftarrow x$
        $x \leftarrow T(x)$ ▷ *One iteration*
    until $\|x - x_{\text{old}}\| \leq \varepsilon$
    return x
end procedure
for all $x_0 \in S$ **do**
    REFINE(x_0, ε)
end for

Math rule: put variables and expressions in `$...$`. Ordinary prose stays outside math mode. The examples illustrate notation and control flow, not a convergence guarantee.

PORTABILITY / ORDINARY LATEX

Use the same code outside LyX

The file `demo/pseudocode-demo.tex` is a complete standalone LaTeX document containing the three examples. Compile it with pdfLaTeX. No Codex-specific command or custom document class is required.

A complete minimal document

```
\documentclass{article}
\usepackage{amsmath,amssymb}
\usepackage{algorithm}
\usepackage[noEnd=false]{algpseudocodex}
\begin{document}
\begin{algorithm}
\caption{A small counter}\label{alg:counter}
\begin{algorithmic}[1]
\State $k\gets 0$
\While{$k<3$}
  \State $k\gets k+1$
\EndWhile
\State \Return $k$
\end{algorithmic}
\end{algorithm}
See Algorithm~\ref{alg:counter}.
\end{document}
```

Captions, labels and macros

In LyX, insert a native **Algorithm float**, give it a caption and label, then place the pseudocode Preview block in the float body. Keep the float itself outside the Preview block. In standalone LaTeX, use the `algorithm` environment shown above.

Use your own math macros in the source. Preamble definitions are available to both preview and export; this feature also carries native LyX math-macro definitions into the raw-code preview. Keep each definition before the algorithm that uses it. For raw-code math, explicitly load packages such as `amsmath` and `amssymb` in Document Settings; LyX cannot infer all package requirements from raw LaTeX.

Package settings: a new document receives `\usepackage[noEnd=false]{algpseudocodex}` when needed. An existing `algpseudocodex` or `algpseudocode` setup is retained. The default `algpseudocodex` setting hides end statements; `noEnd=false` shows them, as in this guide.

REFERENCE / EDITING AND RECOVERY

Working with the new environments

Entries in the env dropdown

Entry	What it inserts or does
LaTeX Pseudocode	Creates a complete editable preview block.
LaTeX While / For / For All	Inserts paired loop commands.
LaTeX If / Else	Inserts a conditional with both branches.
LaTeX Repeat / Until	Inserts a loop with its final condition.
LaTeX Function / Procedure	Inserts a named block and its end.
LaTeX State / Return / Comment	Inserts a statement, return, or comment.
LaTeX Input / Output	Inserts Require and Ensure declarations.
Render LaTeX Pseudocode	Leaves the source and refreshes its preview.

Outside a source block, a template creates a new wrapped preview. Inside the raw LaTeX source, it inserts commands at the caret without adding another wrapper. A template is editable example code: replace its conditions, variable names and statements. Clear a selection before inserting a template.

From an Enumerate algorithm

Copy the mathematical content into `\State` commands. Replace typed While/For/If keywords with the paired LaTeX commands, and remove manual `\tab` or spacing macros. Preserve the algorithm's caption and label. Check branch boundaries while converting; the editor does not infer them from whitespace.

If a preview does not appear

- First click *outside* the preview block. While the cursor is inside, LyX intentionally shows editable source.
- Check unmatched braces, missing math delimiters, and pairs such as While/EndWhile and If/EndIf. Keep only one algorithmic wrapper.
- Verify the TeX installation contains `algpseudocodex` (or `algpseudocode`) and the preview tools. An invalid block remains available as source for correction.
- Use the same package options and macros in preview and final output. Font scale, available width, page breaks and final cross-reference numbers can still differ; preview is not a full-document PDF viewer.

Files supplied with this demo

- `pseudocode-guide.pdf`: this guide.
- `pseudocode-demo.lyx`: editable live examples.
- `pseudocode-demo.tex`: standalone examples.
- `pseudocode-guide/main.tex`: guide source.